

Here are a few approaches to make an agent better at planning:

- Write a better system prompt with more examples.
- Give better descriptions of the tools and their parameters so that the model understands them better.
- Rewrite the functions themselves to make them simpler, such as refactoring a complex function into two simpler functions.
- Use a stronger model. In general, stronger models are better at planning.
- Finetune a model for plan generation.

Function calling. Many model providers offer tool use for their models, effectively turning their models into agents. A tool is a function. Invoking a tool is, therefore, often called *function calling*. Different model APIs work differently, but in general, function calling works as follows:

1. *Create a tool inventory.*

Declare all the tools that you might want a model to use. Each tool is described by its execution entry point (e.g., its function name), its parameters, and its documentation (e.g., what the function does and what parameters it needs).

2. *Specify what tools the agent can use.*

Because different queries might need different tools, many APIs let you specify a list of declared tools to be used per query. Some let you control tool use further by the following settings:

`required`

The model must use at least one tool.

`none`

The model shouldn't use any tool.

`auto`

The model decides which tools to use.

Function calling is illustrated in [Figure 6-10](#). This is written in pseudocode to make it representative of multiple APIs. To use a specific API, please refer to its documentation.

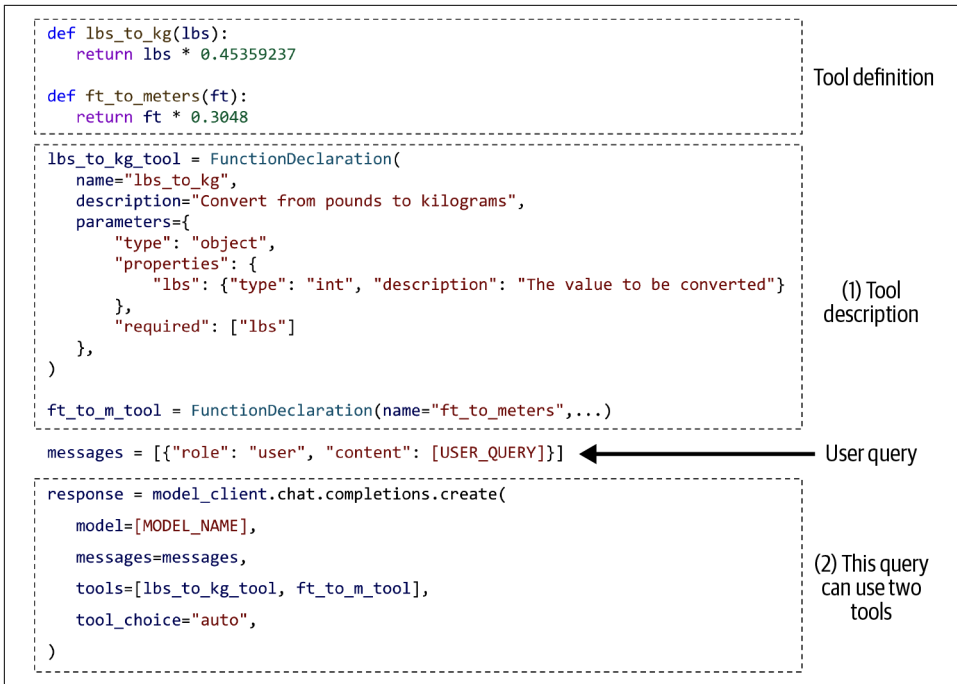


Figure 6-10. An example of a model using two simple tools.

Given a query, an agent defined as in Figure 6-10 will automatically generate what tools to use and their parameters. Some function calling APIs will make sure that only valid functions are generated, though they won't be able to guarantee the correct parameter values.

For example, given the user query "How many kilograms are 40 pounds?", the agent might decide that it needs the tool `lbs_to_kg_tool` with one parameter value of 40. The agent's response might look like this:

```

response = ModelResponse(
    finish_reason='tool_calls',
    message=chat.Message(
        content=None,
        role='assistant',
        tool_calls=[
            ToolCall(
                function=Function(
                    arguments='{"lbs":40}',
                    name='lbs_to_kg'),
                type='function')
        ]
    )
)

```

From this response, you can evoke the function `lbs_to_kg(lbs=40)` and use its output to generate a response to the users.



When working with agents, always ask the system to report what parameter values it uses for each function call. Inspect these values to make sure they are correct.

Planning granularity. A plan is a roadmap outlining the steps needed to accomplish a task. A roadmap can be of different levels of granularity. To plan for a year, a quarter-by-quarter plan is higher-level than a month-by-month plan, which is, in turn, higher-level than a week-to-week plan.

There's a planning/execution trade-off. A detailed plan is harder to generate but easier to execute. A higher-level plan is easier to generate but harder to execute. An approach to circumvent this trade-off is to plan hierarchically. First, use a planner to generate a high-level plan, such as a quarter-to-quarter plan. Then, for each quarter, use the same or a different planner to generate a month-to-month plan.

So far, all examples of generated plans use the exact function names, which is very granular. A problem with this approach is that an agent's tool inventory can change over time. For example, the function to get the current date `get_time()` can be renamed to `get_current_time()`. When a tool changes, you'll need to update your prompt and all your examples. Using the exact function names also makes it harder to reuse a planner across different use cases with different tool APIs.

If you've previously finetuned a model to generate plans based on the old tool inventory, you'll need to finetune the model again on the new tool inventory.

To avoid this problem, plans can also be generated using a more natural language, which is higher-level than domain-specific function names. For example, given the query "What's the price of the best-selling product last week", an agent can be instructed to output a plan that looks like this:

1. get current date
2. retrieve the best-selling product last week
3. retrieve product information
4. generate query
5. generate response

Using more natural language helps your plan generator become robust to changes in tool APIs. If your model was trained mostly on natural language, it'll likely be better at understanding and generating plans in natural language and less likely to hallucinate.

The downside of this approach is that you need a translator to translate each natural language action into executable commands.¹³ However, translating is a much simpler task than planning and can be done by weaker models with a lower risk of hallucination.

Complex plans. The plan examples so far have been sequential: the next action in the plan is *always* executed after the previous action is done. The order in which actions can be executed is called a *control flow*. The sequential form is just one type of control flow. Other types of control flows include the parallel, if statement, and for loop. The following list provides an overview of each control flow, including sequential for comparison:

Sequential

Executing task B after task A is complete, likely because task B depends on task A. For example, the SQL query can be executed only after it's been translated from the natural language input.

Parallel

Executing tasks A and B at the same time. For example, given the query “Find me best-selling products under \$100”, an agent might first retrieve the top 100 best-selling products and, for each of these products, retrieve its price.

If statement

Executing task B or task C depending on the output from the previous step. For example, the agent first checks NVIDIA's earnings report. Based on this report, it can then decide to sell or buy NVIDIA stocks.

For loop

Repeat executing task A until a specific condition is met. For example, keep on generating random numbers until a prime number.

These different control flows are visualized in [Figure 6-11](#).

¹³ Chameleon (Lu et al., 2023) calls this translator a program generator.

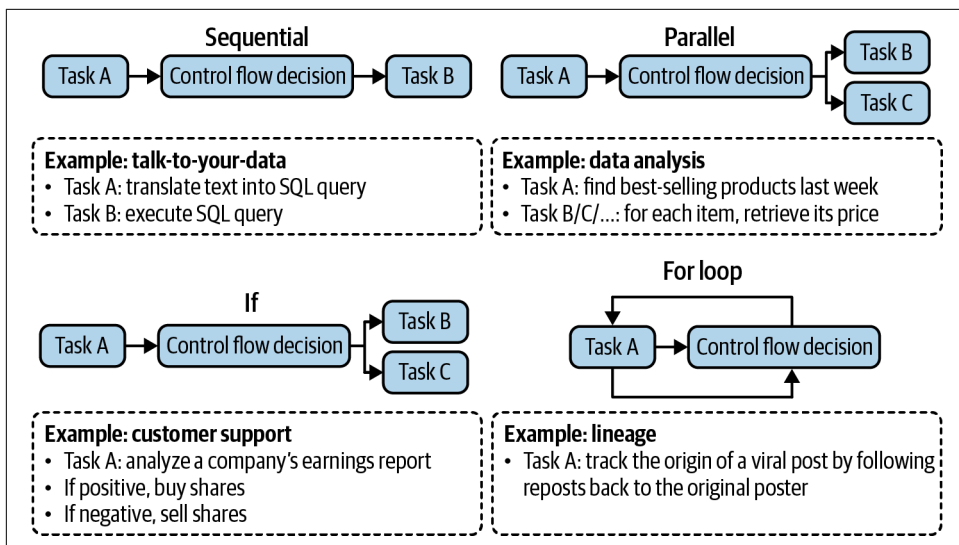


Figure 6-11. Examples of different orders in which a plan can be executed.

In traditional software engineering, conditions for control flows are exact. With AI-powered agents, AI models determine control flows. Plans with non-sequential control flows are more difficult to both generate and translate into executable commands.

When evaluating an agent framework, check what control flows it supports. For example, if the system needs to browse ten websites, can it do so simultaneously? Parallel execution can significantly reduce the latency perceived by users.

Reflection and error correction

Even the best plans need to be constantly evaluated and adjusted to maximize their chance of success. While reflection isn't strictly necessary for an agent to operate, it's necessary for an agent to succeed.

Reflection can be useful in many places during a task process:

- After receiving a user query to evaluate if the request is feasible.
- After the initial plan generation to evaluate whether the plan makes sense.
- After each execution step to evaluate if it's on the right track.
- After the whole plan has been executed to determine if the task has been accomplished.

Reflection and error correction are two different mechanisms that go hand in hand. Reflection generates insights that help uncover errors to be corrected.

Reflection can be done with the same agent using self-critique prompts. It can also be done with a separate component, such as a specialized scorer: a model that outputs a concrete score for each outcome.

First proposed by ReAct (Yao et al., 2022), interleaving reasoning and action has become a common pattern for agents. Yao et al. used the term “reasoning” to encompass both planning and reflection. At each step, the agent is asked to explain its thinking (planning), take actions, then analyze observations (reflection), until the task is considered finished by the agent. The agent is typically prompted, using examples, to generate outputs in the following format:

Thought 1: ...

Act 1: ...

Observation 1: ...

... [continue until reflection determines that the task is finished] ...

Thought N: ...

Act N: Finish [Response to query]

Figure 6-12 shows an example of an agent following the ReAct framework responding to a question from HotpotQA (Yang et al., 2018), a benchmark for multi-hop question answering.

You can implement reflection in a multi-agent setting: one agent plans and takes actions, and another agent evaluates the outcome after each step or after a number of steps.¹⁴

If the agent’s response failed to accomplish the task, you can prompt the agent to reflect on why it failed and how to improve. Based on this suggestion, the agent generates a new plan. This allows agents to learn from their mistakes. For example, given a coding generation task, an evaluator might evaluate that the generated code fails ⅓ of test cases. The agent then reflects the reason it failed is because it didn’t take into account arrays where all numbers are negative. The actor then generates new code, taking into account all-negative arrays.

¹⁴ This reminds me of the actor-critic (AC) agent method (Konda and Tsitsiklis, 1999) in reinforcement learning.

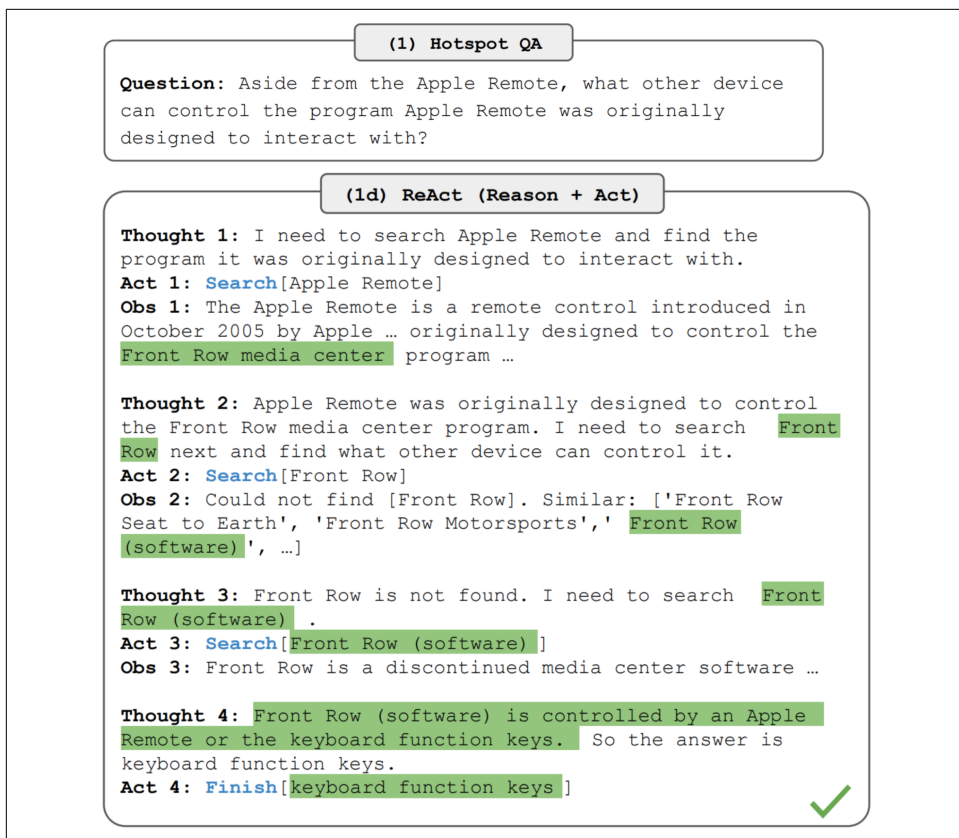


Figure 6-12. A ReAct agent in action. Image from the ReAct paper (Yao et al., 2022). The image is licensed under CC BY 4.0.

This is the approach that Reflexion (Shinn et al., 2023) took. In this framework, reflection is separated into two modules: an evaluator that evaluates the outcome and a self-reflection module that analyzes what went wrong. Figure 6-13 shows examples of Reflexion agents in action. The authors used the term “trajectory” to refer to a plan. At each step, after evaluation and self-reflection, the agent proposes a new trajectory.

Compared to plan generation, reflection is relatively easy to implement and can bring surprisingly good performance improvement. The downside of this approach is latency and cost. Thoughts, observations, and sometimes actions can take a lot of tokens to generate, which increases cost and user-perceived latency, especially for tasks with many intermediate steps. To nudge their agents to follow the format, both ReAct and Reflexion authors used plenty of examples in their prompts. This increases the cost of computing input tokens and reduces the context space available for other information.

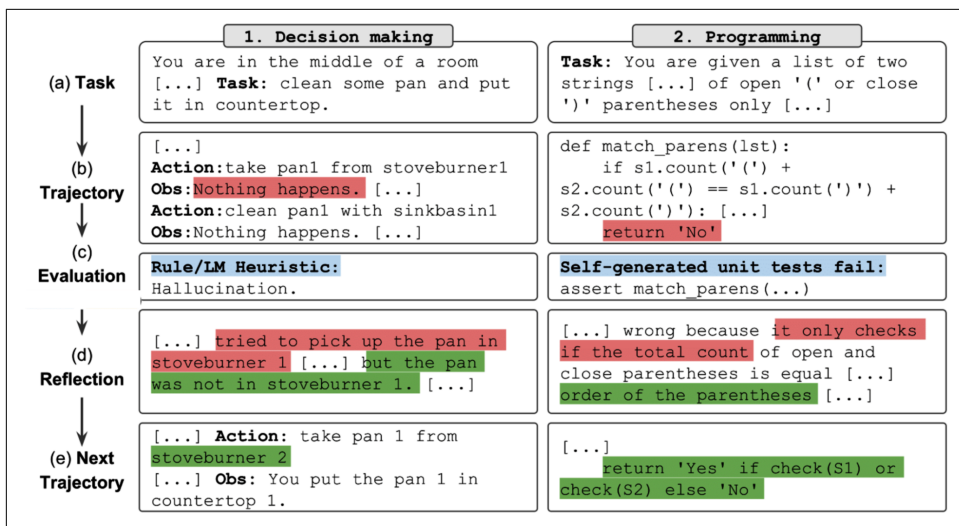


Figure 6-13. Examples of how Reflexion agents work. Images from the *Reflexion GitHub repo*.

Tool selection

Because tools often play a crucial role in a task's success, tool selection requires careful consideration. The tools to give your agent depend on the environment and the task, but they also depend on the AI model that powers the agent.

There's no foolproof guide on how to select the best set of tools. Agent literature consists of a wide range of tool inventories. For example, Toolformer (Schick et al., 2023) finetuned GPT-J to learn five tools. Chameleon (Lu et al., 2023) uses 13 tools. On the other hand, Gorilla (Patil et al., 2023) attempted to prompt agents to select the right API call among 1,645 APIs.

More tools give the agent more capabilities. However, the more tools there are, the harder it is to efficiently use them. It's similar to how it's harder for humans to master a large set of tools. Adding tools also means increasing tool descriptions, which might not fit into a model's context.

Like many other decisions while building AI applications, tool selection requires experimentation and analysis. Here are a few things you can do to help you decide:

- Compare how an agent performs with different sets of tools.
- Do an ablation study to see how much the agent's performance drops if a tool is removed from its inventory. If a tool can be removed without a performance drop, remove it.

- Look for tools that the agent frequently makes mistakes on. If a tool proves too hard for the agent to use—for example, extensive prompting and even finetuning can't get the model to learn to use it—change the tool.
- Plot the distribution of tool calls to see what tools are most used and what tools are least used. **Figure 6-14** shows the differences in tool use patterns of GPT-4 and ChatGPT in Chameleon (Lu et al., 2023).

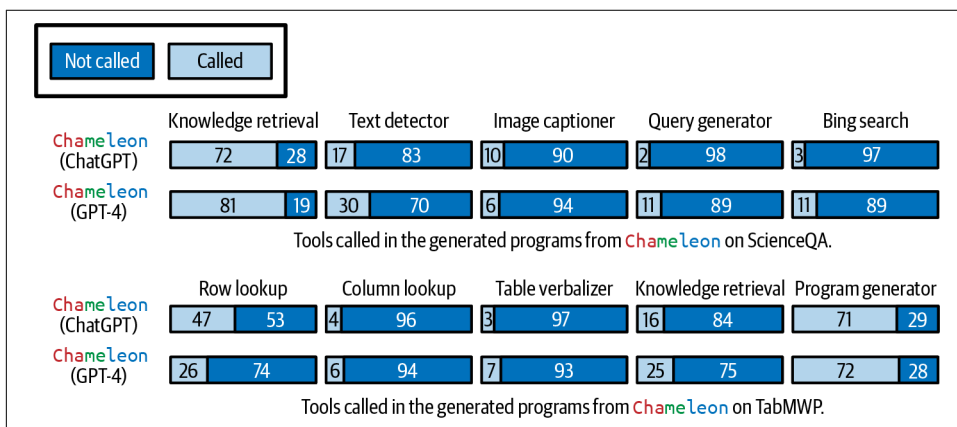


Figure 6-14. Different models and tasks express different tool use patterns. Image from Lu et al. (2023). Adapted from an original image licensed under CC BY 4.0.

Experiments by Lu et al. (2023) also demonstrate two points:

1. Different tasks require different tools. ScienceQA, the science question answering task, relies much more on knowledge retrieval tools than TabMWP, a tabular math problem-solving task.
2. Different models have different tool preferences. For example, GPT-4 seems to select a wider set of tools than ChatGPT. ChatGPT seems to favor image captioning, while GPT-4 seems to favor knowledge retrieval.



When evaluating an agent framework, evaluate what planners and tools it supports. Different frameworks might focus on different categories of tools. For example, AutoGPT focuses on social media APIs (Reddit, X, and Wikipedia), whereas Composio focuses on enterprise APIs (Google Apps, GitHub, and Slack).

As your needs will likely change over time, evaluate how easy it is to extend your agent to incorporate new tools.

As humans, we become more productive not just by using the tools we're given, but also by creating progressively more powerful tools from simpler ones. Can AI create new tools from its initial tools?

Chameleon (Lu et al., 2023) proposes the study of tool transition: after tool X, how likely is the agent to call tool Y? **Figure 6-15** shows an example of tool transition. If two tools are frequently used together, they can be combined into a bigger tool. If an agent is aware of this information, the agent itself can combine initial tools to continually build more complex tools.

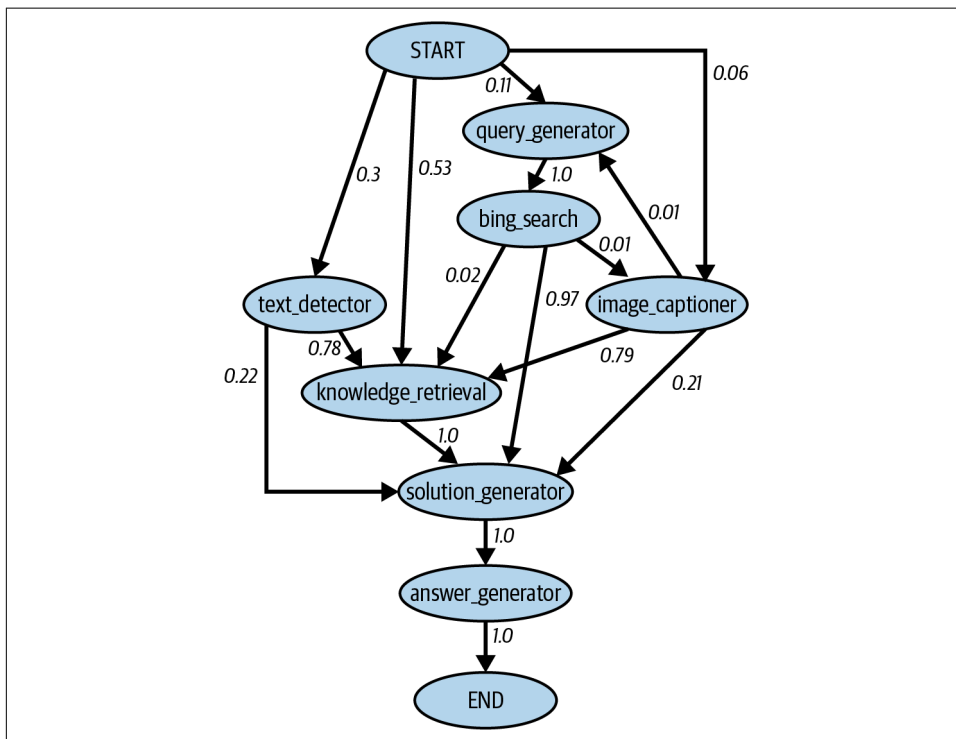


Figure 6-15. A tool transition tree by Lu et al. (2023). Adapted from an original image licensed under CC BY 4.0.

Vogager (Wang et al., 2023) proposes a skill manager to keep track of new skills (tools) that an agent acquires for later reuse. Each skill is a coding program. When the skill manager determines a newly created skill is to be useful (e.g., because it's successfully helped an agent accomplish a task), it adds this skill to the skill library (conceptually similar to the tool inventory). This skill can be retrieved later to use for other tasks.